

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 10. List

Lecture: 104. List Concatenation & Repetition

Section 1: Lecture Summary

This lecture focuses on various **operations on Python lists**, primarily covering **concatenation** and **comparison**, along with some other common operations like repetition and membership testing. It explains how these operations work similarly to string operations but are applicable to lists too. The instructor demonstrates these operations using a Jupyter notebook with live coding examples showing concatenation using the `+` operator and `extend()` method, repetition with `*`, membership testing using `in` and `not in`, and list comparison using relational operators (`==`, `!=`, `<`, `>`, etc.). Key distinctions such as the immutability of the original lists when using concatenation (`+`) versus modification in place with `extend()` are emphasized. The lecture further explains how comparisons between lists are conducted element-wise and how Python handles nested lists in membership tests.

Section 2: Key Concepts and Explanations

- Concatenation (`+` operator and `extend()` method)

- Using `+` combines two lists into a new list without modifying the originals.
- Attempting to concatenate a list with a non-list type (e.g., integer) raises a `TypeError`.
- Using `extend()` appends elements of one list to another, modifying the list in place.

- Repetition (`*` operator)

- Multiplying a list by an integer repeats its elements to form a new list.
- The repetition count must be an integer; floats cause a `TypeError`.

- Membership Operations (`in`, `not in`)

- Checks whether an element exists within a list.
- Works on nested lists to check if a sublist exists as an element.
- Commonly used within `for` loops for iteration over list elements.

- List Comparison

- Comparison operators such as `==`, `!=`, `<`, `>`, `<=`, `>=` compare lists element-wise.

- Equality (`==`) requires both lists to have the same elements in the same order.
- Inequality (`!=`) returns `True` if any element differs or list lengths differ.
- Ordering comparisons (`<`, `>`) compare elements lexicographically, not by length.
- Important for data science applications where datasets pulled from various sources are compared.

Section 3: Example Code and Use Cases

Concatenation using '+' operator

Attempting to concatenate list with integer raises `TypeError`

`L1 + 4` Uncommenting raises `TypeError`

Concatenating list with a list containing one element

Concatenation with `extend()` method modifies the list in place

Repetition of a list

Multiplying by non-integer causes error

`L2 = L1 * 2.5` Uncommenting raises `TypeError`

Membership operations

Membership with nested lists

Using membership in for loop

Output:

red

green

blue

List comparison (equality and inequality)

Ordering comparisons

```
L1 = [1, 2, 3]
L2 = [8, 9, 10]
L3 = L1 + L2
print(L3) # Output: [1, 2, 3, 8, 9, 10]
```

```

print(L1 + [4]) # Output: [1, 2, 3, 4]
print(L1)       # Original list unchanged: [1, 2, 3]

L1.extend([4, 5, 6])
print(L1) # Output: [1, 2, 3, 4, 5, 6]

L1 = [1, 2, 3]
L2 = L1 * 3
print(L2) # Output: [1, 2, 3, 1, 2, 3, 1, 2, 3]

L1 = [3, 4, 5]
print(3 in L1) # Output: True
print(7 in L1) # Output: False

NL1 = [[1, 2], [3, 4], 5]
print(3 in NL1) # Output: False (3 not directly in list)
print([3, 4] in NL1) # Output: True (sublist exists)

colors = ['red', 'green', 'blue']
for color in colors:
    print(color)

L1 = [1, 2, 3]
L2 = [1, 2, 3]
L3 = [3, 2, 1]

print(L1 == L2) # True
print(L1 == L3) # False
print(L2 != L3) # True

L1 = [1, 2, 3]
L2 = [1, 2, 3, 4]
L3 = [1, 2, 1]

print(L1 > L2) # False (L2 has extra element making it greater)
print(L1 < L2) # True
print(L1 > L3) # True (third element 3 > 1)

```

Section 4: Key Takeaways

- The `+` operator concatenates two lists and returns a new list, leaving originals unchanged.
- The `extend()` method concatenates by adding elements to the existing list, modifying it.
- List repetition with `` requires an integer; the resulting list repeats the original elements.

- Membership operators ``in`` and ``not in`` return Booleans indicating element presence, working even with nested lists.
- Lists can be compared element-wise using relational operators; equality means all elements and order match.
- Ordering comparisons compare elements lexicographically, not by length.
- Understanding these operations is important for practical Python programming and data manipulation tasks.